```
# ====================================
# Import libraries
# ====================================
import torch
from torch import nn, optim
import torch.nn.functional as F
from torch.utils.data import random_split, DataLoader
import torchvision.transforms.v2 as T
from torch.utils.data import TensorDataset
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import random

"""
This code has been used to test different possible CNNs settings.
We tried different depths (Experiment 1), initial weights (Experiment 2), learning rates (Experiment 3),
number of epochs (Experiment 4), and batch sizes (Experiment 5).
The best CNN found with this code, has then been used for all the 3 parts of task2.
"""
```

```
'\nThis code has been used to test different possible CNNs settings.\nWe tried different depths (Experiment 1), initial weights (Exper
iment 2), learning rates (Experiment 3),\nnumber of epochs (Experiment 4), and batch sizes (Experiment 5).\nThe best CNN found with th
is code, has then been used for all the 3 parts of task2.\n'
```

```
#Check whether a GPU is available
print("GPU available:", torch.cuda.is_available())

#Check which GPU is being used
if torch.cuda.is_available():
    print("GPU name:", torch.cuda.get_device_name(0))
else:
    print("Using CPU")
```

```
GPU available: True
GPU name: Tesla T4
```

```
# ====================================
# Reproducibility and device
# ====================================
r_seed = 99
generator = torch.Generator()

def reset_seed(seed=r_seed):
    """Reset all seeds at once."""
    random.seed(seed) #reset python random seed
    np.random.seed(seed) #reset numpy random seed
    torch.manual_seed(seed) #reset pytorch random seed
    generator.manual_seed(seed) #reset the dataloader random seed
    if torch.cuda.is_available(): #if we use the gpu
        torch.cuda.manual_seed_all(seed)

reset_seed()

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
Using device: cuda
```

```
#This cell runs on Google Colab if you save the data files in a folder called "data" in Drive.
#To run locally, skip this cell and set the paths to data/075_images.npy

from google.colab import drive
drive.mount('/content/drive')
```

```
# ====================================
# Load the Clock dataset (75x75)
# ====================================
images_path = '/content/drive/MyDrive/DL_Assignment_1/data/075_images.npy'
labels_path = '/content/drive/MyDrive/DL_Assignment_1/data/075_labels.npy'

images_np = np.load(images_path)
labels_np = np.load(labels_path)

#Check out our images
print("images:", images_np.shape, images_np.dtype, images_np.min(), images_np.max()) #18'000 images, 75x75
print("labels:", labels_np.shape, labels_np.dtype) #18'000 labels, 2 dimensions (hour, min)
```

```
print("hours:", np.unique(labels_np[:, 0])) #they go from 0 to 11
print("minutes:", labels_np[:, 1].min(), labels_np[:, 1].max()) #they go from 0 to 59

#Convert into pytorch tensors
images = torch.from_numpy(images_np).float() #turn numpy array into pytorch tensor, float
images = images.unsqueeze(1) #add channel dimension, we need format (batch_size, channels, height, width) (greyscale = 1 channel)
mean = images.mean()
std = images.std()
images = (images - mean) / std #standardize the pixels (easier for training)

labels_minutes = torch.from_numpy(labels_np[:, 0] * 60 + labels_np[:, 1]).long() #hours*60 + minutes, and turn it into pytorch tensor
print(labels_minutes.shape, labels_minutes.min(), labels_minutes.max()) #18'000 labels, 1 dimension (min). Minutes go from 0 (00:00) t
```

```
images: (18000, 75, 75) uint8 16 255
labels: (18000, 2) int64
hours: [ 0  1  2  3  4  5  6  7  8  9 10 11]
minutes: 0 59
torch.Size([18000]) tensor(0) tensor(719)
```

```
#====================================
# Train-Validation-Test split
# ====================================
reset_seed()

dataset = TensorDataset(images, labels_minutes)
n = len(dataset)

train_size = int(0.8 * n)
valid_size = int(0.1 * n)
test_size = n - train_size - valid_size

train_data, valid_data, test_data = random_split(
    dataset,
    [train_size, valid_size, test_size],
    generator=generator
)

batch_size = 64

train_dataloader = DataLoader(
    train_data,
    batch_size=batch_size,
    shuffle=True,
    generator=generator
)

valid_dataloader = DataLoader(
    valid_data,
    batch_size=batch_size,
    shuffle=True
)

test_dataloader = DataLoader(
    test_data,
    batch_size=batch_size,
    shuffle=True
)
```

```
# ======================================================
# Defining the functions for training and evaluation
# ======================================================


#Train the model

def train_model(model, train_dataloader, optimizer, criterion, epochs, bin_size):

    """Train a model for a given number of epochs."""

    for epoch in range(epochs):

        model.train()
        running_loss = 0.0

        for inputs, minutes in train_dataloader:

            #data agumentation
            #inputs = train_transform(inputs)

            #To use external GPU
            inputs = inputs.to(device)
```

```
            minutes = minutes.to(device)

            #convert the labels from labels in minutes (e.g., 01:30 --> 90 minutes) to labels in classes (e.g., class 3)
            labels_class = (minutes // bin_size).long()

            #zero the parameter gradients
            optimizer.zero_grad()

            #forward + backward + optimize
            outputs = model(inputs) #predictions
            loss = criterion(outputs, labels_class) #compute loss
            loss.backward() #compute gradients
            optimizer.step() #optimize weights

            #compute running loss
            running_loss += loss.item()
        print(
            f"Epoch {epoch + 1}/{epochs}: "
            f"- Loss = {running_loss/len(train_dataloader):.4f}"
        )

#Validate the model

def evaluate_model(model, dataloader, bin_size):

    """Evaluate a model using the average circular time error."""

    model.eval()

    total_error = 0
    total = 0

    with torch.no_grad():
        for inputs, minutes in dataloader:

            #To use external GPU
            inputs = inputs.to(device)
            minutes = minutes.to(device)

            #Get predictions and classes
            outputs = model(inputs) #predictions (in classes!!!)
            predictions = outputs.argmax(dim=1) #assign to the class with the highest score

            #Convert predicted classes back into total minutes
            predicted_minutes = predictions * bin_size + (bin_size - 1) / 2 #midpoint to represent the class

            #Calculate absolute difference and the circular difference
            difference = torch.abs(predicted_minutes - minutes)
            circular_difference = torch.minimum(difference, 720 - difference)

            #Compute the errors
            total_error += circular_difference.sum().item()
            total += minutes.size(0)

    common_sense_error = total_error / total

    return common_sense_error
```

```
# ====================================
# Trying out different CNNs
# ====================================

class Net_2conv(nn.Module):   #baseline: task1_CNN, plus padding=1, and one less fully connected layer
    def __init__(self, num_classes):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 3, padding=1)
        self.fc1 = nn.Linear(16 * 18 * 18, 120)
        self.fc2 = nn.Linear(120, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x


class Net_3conv(nn.Module):   #Add one convolutional layer
```

```
    def __init__(self, num_classes):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 3, padding=1)
        self.conv3 = nn.Conv2d(16, 32, 3, padding=1)
        self.fc1 = nn.Linear(32 * 9 * 9, 120)
        self.fc2 = nn.Linear(120, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x


class Net_4conv(nn.Module):   #Add another convolutional layer
    def __init__(self, num_classes):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 3, padding=1)
        self.conv3 = nn.Conv2d(16, 32, 3, padding=1)
        self.conv4 = nn.Conv2d(32, 64, 3, padding=1)
        self.fc1 = nn.Linear(64 * 4 * 4, 120)
        self.fc2 = nn.Linear(120, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        x = self.pool(F.relu(self.conv4(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

```
# ====================================
# Trying out different initial weights
# ====================================
def init_he(module):
    """He weight initialisation""" #this is made for ReLU
    if isinstance(module, (nn.Linear, nn.Conv2d)):
        nn.init.kaiming_normal_(module.weight, nonlinearity="relu")
        nn.init.zeros_(module.bias) #biases start at 0

def init_xavier(module):
    """Xavier (Glorot) initialisation""" #this is what the book introduced (dive into deep learning)
    if isinstance(module, (nn.Linear, nn.Conv2d)):
        nn.init.xavier_uniform_(module.weight)
        nn.init.zeros_(module.bias) #biases start at 0
```

```
# ====================================
# Run experiment function
# ====================================

#One function to be able to run the different experiments by changing one setting at a time
def run_experiment(NetClass, seed, init_fn=None, bin_size=30, epochs=20, lr=0.001, batch_size=64):

    #New seed for each experiment
    reset_seed(seed)

    #The training loader is built here, so that we can change the batch size
    train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True, generator=generator)

    num_classes = 720 // bin_size
    model = NetClass(num_classes).to(device)

    #Initialization weights
    if init_fn is not None:
        model.apply(init_fn)

    #Loss function
    criterion = nn.CrossEntropyLoss()

    #Optimizer
    optimizer = optim.Adam(model.parameters(), lr=lr)
```

```
    #Train the model
    train_model(model, train_loader, optimizer, criterion, epochs, bin_size)

    #Validate the model
    return evaluate_model(model, valid_dataloader, bin_size)
```

```
# ====================================
# Experiment 1: depth of the CNN
# ====================================
nets = {"Net_2conv": Net_2conv, "Net_3conv": Net_3conv, "Net_4conv": Net_4conv}

#We run each experiment is run 3 times, so that the results are not due to random luck
seeds = [1, 2, 3]

rows = []
for name, NetClass in nets.items():
    for seed in seeds:
        err = run_experiment(NetClass, seed)
        rows.append({"net": name, "seed": seed, "common_sense_error": err})

depth_df = pd.DataFrame(rows)
print(depth_df)
print(depth_df.groupby("net")["common_sense_error"].agg(["mean", "std"]))
```

```
Epoch 1/20: - Loss = 3.1798
Epoch 2/20: - Loss = 2.9807
Epoch 3/20: - Loss = 2.4097
Epoch 4/20: - Loss = 2.0793
Epoch 5/20: - Loss = 1.8681
Epoch 6/20: - Loss = 1.7314
Epoch 7/20: - Loss = 1.5817
Epoch 8/20: - Loss = 1.4692
Epoch 9/20: - Loss = 1.3723
Epoch 10/20: - Loss = 1.2648
Epoch 11/20: - Loss = 1.1793
Epoch 12/20: - Loss = 1.0892
Epoch 13/20: - Loss = 1.0302
Epoch 14/20: - Loss = 0.9640
Epoch 15/20: - Loss = 0.9033
Epoch 16/20: - Loss = 0.8506
Epoch 17/20: - Loss = 0.8125
Epoch 18/20: - Loss = 0.7755
Epoch 19/20: - Loss = 0.7221
Epoch 20/20: - Loss = 0.6723
Epoch 1/20: - Loss = 3.1760
Epoch 2/20: - Loss = 2.8421
Epoch 3/20: - Loss = 2.1747
Epoch 4/20: - Loss = 1.7452
Epoch 5/20: - Loss = 1.4234
Epoch 6/20: - Loss = 1.1595
Epoch 7/20: - Loss = 0.9507
Epoch 8/20: - Loss = 0.7766
Epoch 9/20: - Loss = 0.6329
Epoch 10/20: - Loss = 0.5212
Epoch 11/20: - Loss = 0.4268
Epoch 12/20: - Loss = 0.3454
Epoch 13/20: - Loss = 0.2965
Epoch 14/20: - Loss = 0.2475
Epoch 15/20: - Loss = 0.1912
Epoch 16/20: - Loss = 0.1793
Epoch 17/20: - Loss = 0.1359
Epoch 18/20: - Loss = 0.1105
Epoch 19/20: - Loss = 0.1205
Epoch 20/20: - Loss = 0.1226
Epoch 1/20: - Loss = 3.1730
Epoch 2/20: - Loss = 2.8351
Epoch 3/20: - Loss = 2.2903
Epoch 4/20: - Loss = 2.0472
Epoch 5/20: - Loss = 1.8492
Epoch 6/20: - Loss = 1.7161
Epoch 7/20: - Loss = 1.6019
Epoch 8/20: - Loss = 1.4880
Epoch 9/20: - Loss = 1.3833
Epoch 10/20: - Loss = 1.2947
Epoch 11/20: - Loss = 1.2158
Epoch 12/20: - Loss = 1.1452
Epoch 13/20: - Loss = 1.0842
Epoch 14/20: - Loss = 1.0215
Epoch 15/20: - Loss = 0.9499
Epoch 16/20: - Loss = 0.9119
Epoch 17/20: - Loss = 0.8774
Epoch 18/20: - Loss = 0.8244
```

```
# ====================================
# Experiment 2: initial weights (on Net_4conv)
# ====================================
inits = {"default": None, "xavier": init_xavier, "he": init_he}
seeds = [1, 2, 3]

rows = []
for init_name, init_fn in inits.items():
    for seed in seeds:
        err = run_experiment(Net_4conv, seed, init_fn=init_fn)
        rows.append({"init": init_name, "seed": seed, "common_sense_error": err})

init_df = pd.DataFrame(rows)
print(init_df)
print(init_df.groupby("init")["common_sense_error"].agg(["mean", "std"]))
```

```
Epoch 16/20: - Loss = 0.1978
Epoch 17/20: - Loss = 0.1622
Epoch 18/20: - Loss = 0.1406
Epoch 19/20: - Loss = 0.1409
Epoch 20/20: - Loss = 0.1239
Epoch 1/20: - Loss = 3.1767
Epoch 2/20: - Loss = 2.7345
Epoch 3/20: - Loss = 1.9452
Epoch 4/20: - Loss = 1.4157
Epoch 5/20: - Loss = 1.0518
Epoch 6/20: - Loss = 0.8066
Epoch 7/20: - Loss = 0.6417
Epoch 8/20: - Loss = 0.5208
Epoch 9/20: - Loss = 0.4356
Epoch 10/20: - Loss = 0.3613
Epoch 11/20: - Loss = 0.3114
Epoch 12/20: - Loss = 0.2680
Epoch 13/20: - Loss = 0.2286
Epoch 14/20: - Loss = 0.2193
Epoch 15/20: - Loss = 0.1548
Epoch 16/20: - Loss = 0.1784
Epoch 17/20: - Loss = 0.1250
Epoch 18/20: - Loss = 0.1122
Epoch 19/20: - Loss = 0.1766
Epoch 20/20: - Loss = 0.1175
Epoch 1/20: - Loss = 3.2302
Epoch 2/20: - Loss = 3.1710
Epoch 3/20: - Loss = 2.9878
Epoch 4/20: - Loss = 2.5335
Epoch 5/20: - Loss = 2.1350
Epoch 6/20: - Loss = 1.8938
Epoch 7/20: - Loss = 1.6988
Epoch 8/20: - Loss = 1.5255
Epoch 9/20: - Loss = 1.3803
Epoch 10/20: - Loss = 1.2419
Epoch 11/20: - Loss = 1.1399
Epoch 12/20: - Loss = 1.0546
Epoch 13/20: - Loss = 0.9957
Epoch 14/20: - Loss = 0.9222
Epoch 15/20: - Loss = 0.8569
Epoch 16/20: - Loss = 0.7906
Epoch 17/20: - Loss = 0.7524
Epoch 18/20: - Loss = 0.7074
Epoch 19/20: - Loss = 0.6489
Epoch 20/20: - Loss = 0.6216
Epoch 1/20: - Loss = 3.2265
Epoch 2/20: - Loss = 2.8453
Epoch 3/20: - Loss = 2.2453
Epoch 4/20: - Loss = 1.8358
Epoch 5/20: - Loss = 1.4963
Epoch 6/20: - Loss = 1.2187
Epoch 7/20: - Loss = 0.9973
Epoch 8/20: - Loss = 0.8135
Epoch 9/20: - Loss = 0.6936
Epoch 10/20: - Loss = 0.5773
Epoch 11/20: - Loss = 0.5280
Epoch 12/20: - Loss = 0.4415
Epoch 13/20: - Loss = 0.3790
```

```
# ====================================
# Experiment 3: learning rate (on Net_4conv, default weights)
# ====================================

#Here we use only one seed because in early runs we saw that the differences were large enough to be distinguished without
#averaging over 3 seeds' results.
#Here we use again more epochs (40), because smaller learning rates learn more slowly, and might need more epochs

seed = 1 #one fixed seed for every learning rate

rows = []
```

```
for lr in [0.003, 0.001, 0.0003]:
    err = run_experiment(Net_4conv, seed, init_fn=None, epochs=40, lr=lr)
    rows.append({"lr": lr, "seed": seed, "common_sense_error": err})
    print(f"lr={lr}: {err:.2f} min")

lr_df = pd.DataFrame(rows)
print(lr_df)
```

```
Epoch 20/40: - Loss = 0.2344
Epoch 21/40: - Loss = 0.2347
Epoch 22/40: - Loss = 0.2240
Epoch 23/40: - Loss = 0.2112
Epoch 24/40: - Loss = 0.2026
Epoch 25/40: - Loss = 0.1861
Epoch 26/40: - Loss = 0.1787
Epoch 27/40: - Loss = 0.1714
Epoch 28/40: - Loss = 0.1696
Epoch 29/40: - Loss = 0.1628
Epoch 30/40: - Loss = 0.1375
Epoch 31/40: - Loss = 0.1464
Epoch 32/40: - Loss = 0.1840
Epoch 33/40: - Loss = 0.1218
Epoch 34/40: - Loss = 0.1125
Epoch 35/40: - Loss = 0.1375
Epoch 36/40: - Loss = 0.1655
Epoch 37/40: - Loss = 0.1349
Epoch 38/40: - Loss = 0.0786
Epoch 39/40: - Loss = 0.0978
Epoch 40/40: - Loss = 0.0959
lr=0.001: 14.44 min
Epoch 1/40: - Loss = 3.1791
Epoch 2/40: - Loss = 3.1740
Epoch 3/40: - Loss = 2.9834
Epoch 4/40: - Loss = 2.4880
Epoch 5/40: - Loss = 2.1410
Epoch 6/40: - Loss = 1.9183
Epoch 7/40: - Loss = 1.7380
Epoch 8/40: - Loss = 1.5843
Epoch 9/40: - Loss = 1.4452
Epoch 10/40: - Loss = 1.3236
Epoch 11/40: - Loss = 1.2006
Epoch 12/40: - Loss = 1.1025
Epoch 13/40: - Loss = 1.0146
Epoch 14/40: - Loss = 0.9358
Epoch 15/40: - Loss = 0.8572
Epoch 16/40: - Loss = 0.8081
Epoch 17/40: - Loss = 0.7568
Epoch 18/40: - Loss = 0.7067
Epoch 19/40: - Loss = 0.6599
Epoch 20/40: - Loss = 0.6215
Epoch 21/40: - Loss = 0.5843
Epoch 22/40: - Loss = 0.5570
Epoch 23/40: - Loss = 0.5245
Epoch 24/40: - Loss = 0.4878
Epoch 25/40: - Loss = 0.4679
Epoch 26/40: - Loss = 0.4380
Epoch 27/40: - Loss = 0.4155
Epoch 28/40: - Loss = 0.3966
Epoch 29/40: - Loss = 0.3809
Epoch 30/40: - Loss = 0.3705
Epoch 31/40: - Loss = 0.3599
Epoch 32/40: - Loss = 0.3417
Epoch 33/40: - Loss = 0.3192
Epoch 34/40: - Loss = 0.2926
Epoch 35/40: - Loss = 0.2929
Epoch 36/40: - Loss = 0.2932
Epoch 37/40: - Loss = 0.2678
```

```
# ====================================
# Experiment 4: number of epochs and plot the learning curves
# ====================================

#We use Net_4conv, default weights, lr = 0.001, 30-minute bins, 1 seed
#Here we use more epochs because we want to see where we can stop

def train_validate_curves(NetClass, seed, bin_size=30, epochs=50, lr=0.001):
    reset_seed(seed)
    num_classes = 720 // bin_size
    model = NetClass(num_classes).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    train_losses, valid_losses, valid_errors = [], [], []
    for epoch in range(epochs):
        #training
        model.train()
        running_loss = 0.0
```

```
        for inputs, minutes in train_dataloader:
            inputs, minutes = inputs.to(device), minutes.to(device)
            labels_class = (minutes // bin_size).long()
            optimizer.zero_grad()
            loss = criterion(model(inputs), labels_class)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()
        train_losses.append(running_loss / len(train_dataloader))

        #validation loss (no learning here)
        model.eval()
        running_loss = 0.0
        with torch.no_grad():
            for inputs, minutes in valid_dataloader:
                inputs, minutes = inputs.to(device), minutes.to(device)
                labels_class = (minutes // bin_size).long()
                running_loss += criterion(model(inputs), labels_class).item()
        valid_losses.append(running_loss / len(valid_dataloader))

        #validation common-sense error (the metric we really care about)
        valid_errors.append(evaluate_model(model, valid_dataloader, bin_size))

        print(f"Epoch {epoch + 1}/{epochs}: train loss = {train_losses[-1]:.4f}, "
              f"valid loss = {valid_losses[-1]:.4f}, valid error = {valid_errors[-1]:.2f} min")

    return model, train_losses, valid_losses, valid_errors

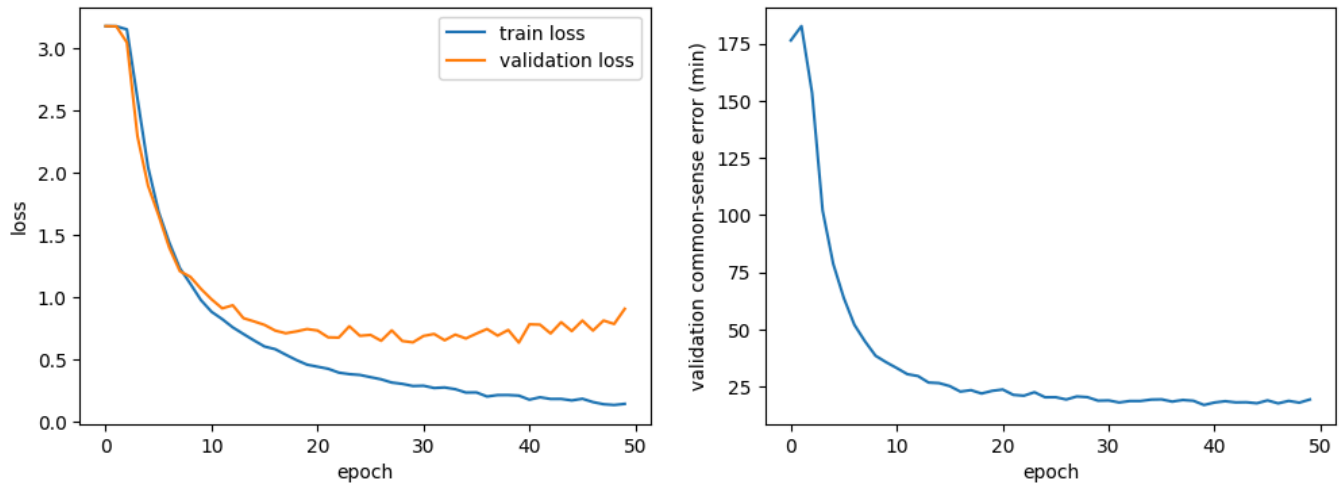

epochs = 50
model, train_losses, valid_losses, valid_errors = train_validate_curves(Net_4conv, seed=1, epochs=epochs)
```

```
Epoch 1/50: train loss = 3.1793, valid loss = 3.1801, valid error = 176.41 min
Epoch 2/50: train loss = 3.1787, valid loss = 3.1781, valid error = 182.66 min
Epoch 3/50: train loss = 3.1533, valid loss = 3.0482, valid error = 153.53 min
Epoch 4/50: train loss = 2.5966, valid loss = 2.2936, valid error = 101.95 min
Epoch 5/50: train loss = 2.0453, valid loss = 1.8958, valid error = 78.66 min
Epoch 6/50: train loss = 1.6852, valid loss = 1.6562, valid error = 63.72 min
Epoch 7/50: train loss = 1.4396, valid loss = 1.3983, valid error = 51.98 min
Epoch 8/50: train loss = 1.2332, valid loss = 1.2090, valid error = 44.75 min
Epoch 9/50: train loss = 1.1062, valid loss = 1.1630, valid error = 38.45 min
Epoch 10/50: train loss = 0.9753, valid loss = 1.0661, valid error = 35.62 min
Epoch 11/50: train loss = 0.8820, valid loss = 0.9820, valid error = 33.08 min
Epoch 12/50: train loss = 0.8233, valid loss = 0.9098, valid error = 30.41 min
Epoch 13/50: train loss = 0.7567, valid loss = 0.9346, valid error = 29.59 min
Epoch 14/50: train loss = 0.7038, valid loss = 0.8312, valid error = 26.74 min
Epoch 15/50: train loss = 0.6520, valid loss = 0.8049, valid error = 26.50 min
Epoch 16/50: train loss = 0.6029, valid loss = 0.7767, valid error = 25.19 min
Epoch 17/50: train loss = 0.5816, valid loss = 0.7310, valid error = 22.79 min
Epoch 18/50: train loss = 0.5371, valid loss = 0.7096, valid error = 23.41 min
Epoch 19/50: train loss = 0.4943, valid loss = 0.7244, valid error = 21.94 min
Epoch 20/50: train loss = 0.4573, valid loss = 0.7435, valid error = 23.11 min
Epoch 21/50: train loss = 0.4406, valid loss = 0.7317, valid error = 23.70 min
Epoch 22/50: train loss = 0.4241, valid loss = 0.6770, valid error = 21.35 min
Epoch 23/50: train loss = 0.3932, valid loss = 0.6747, valid error = 20.99 min
Epoch 24/50: train loss = 0.3808, valid loss = 0.7657, valid error = 22.50 min
Epoch 25/50: train loss = 0.3744, valid loss = 0.6892, valid error = 20.30 min
Epoch 26/50: train loss = 0.3565, valid loss = 0.6964, valid error = 20.32 min
Epoch 27/50: train loss = 0.3392, valid loss = 0.6492, valid error = 19.34 min
Epoch 28/50: train loss = 0.3134, valid loss = 0.7328, valid error = 20.65 min
Epoch 29/50: train loss = 0.3027, valid loss = 0.6474, valid error = 20.34 min
Epoch 30/50: train loss = 0.2854, valid loss = 0.6367, valid error = 18.81 min
Epoch 31/50: train loss = 0.2876, valid loss = 0.6879, valid error = 18.91 min
Epoch 32/50: train loss = 0.2694, valid loss = 0.7041, valid error = 18.03 min
Epoch 33/50: train loss = 0.2737, valid loss = 0.6532, valid error = 18.66 min
Epoch 34/50: train loss = 0.2599, valid loss = 0.6987, valid error = 18.66 min
Epoch 35/50: train loss = 0.2332, valid loss = 0.6676, valid error = 19.29 min
Epoch 36/50: train loss = 0.2337, valid loss = 0.7071, valid error = 19.39 min
Epoch 37/50: train loss = 0.2009, valid loss = 0.7438, valid error = 18.40 min
Epoch 38/50: train loss = 0.2125, valid loss = 0.6907, valid error = 19.12 min
Epoch 39/50: train loss = 0.2128, valid loss = 0.7361, valid error = 18.77 min
Epoch 40/50: train loss = 0.2083, valid loss = 0.6343, valid error = 16.94 min
Epoch 41/50: train loss = 0.1755, valid loss = 0.7813, valid error = 18.01 min
Epoch 42/50: train loss = 0.1948, valid loss = 0.7789, valid error = 18.58 min
Epoch 43/50: train loss = 0.1813, valid loss = 0.7081, valid error = 18.07 min
Epoch 44/50: train loss = 0.1815, valid loss = 0.7986, valid error = 18.12 min
Epoch 45/50: train loss = 0.1694, valid loss = 0.7272, valid error = 17.69 min
Epoch 46/50: train loss = 0.1829, valid loss = 0.8119, valid error = 18.93 min
Epoch 47/50: train loss = 0.1562, valid loss = 0.7311, valid error = 17.64 min
Epoch 48/50: train loss = 0.1390, valid loss = 0.8120, valid error = 18.72 min
Epoch 49/50: train loss = 0.1330, valid loss = 0.7844, valid error = 17.94 min
Epoch 50/50: train loss = 0.1419, valid loss = 0.9061, valid error = 19.34 min
```

```
#Plot
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
```

```
axes[0].plot(train_losses, label="train loss")
axes[0].plot(valid_losses, label="validation loss")
axes[0].set_xlabel("epoch")
axes[0].set_ylabel("loss")
axes[0].legend()
axes[1].plot(valid_errors)
axes[1].set_xlabel("epoch")
axes[1].set_ylabel("validation common-sense error (min)")
plt.show()

#Error after 20, 30, 40 and 50 epochs
for e in [20, 30, 40, 50]:
    print(f"after {e} epochs: validation error = {valid_errors[e - 1]:.2f} minutes")
```




```
after 20 epochs: validation error = 23.11 minutes
after 30 epochs: validation error = 18.81 minutes
after 40 epochs: validation error = 16.94 minutes
after 50 epochs: validation error = 19.34 minutes
```

```
# ====================================
# Experiment 5: batch size
# ====================================

#We want to check if different batch sizes (on Net_4conv, default weights, lr = 0.001) can improve the model.
#Here we go back to using 20 epochs, because hopefully it's enough to see which setting performs best

batch_sizes = [32, 64, 128]
seeds = [1, 2, 3]
rows = []

for batch_size_test in batch_sizes:
    for seed in seeds:
        err = run_experiment(Net_4conv, seed, batch_size=batch_size_test)
        rows.append({"batch_size": batch_size_test, "seed": seed, "common_sense_error": err})

batch_df = pd.DataFrame(rows)
print(batch_df)
print(batch_df.groupby("batch_size")["common_sense_error"].agg(["mean", "std"]))

#NB.
#Batch 32 is slightly better but not enough. Since it would be slower to train, we keep batch 64.
```

```
Epoch 17/20: - Loss = 3.1784
Epoch 18/20: - Loss = 3.1783
Epoch 19/20: - Loss = 3.1783
Epoch 20/20: - Loss = 3.1783
Epoch 1/20: - Loss = 3.1793
```

```
Epoch 17/20: - Loss = 3.1784
```

```
Epoch 18/20: - Loss = 3.1784
Epoch 19/20: - Loss = 3.1783
Epoch 20/20: - Loss = 3.1783
Epoch 1/20: - Loss = 3.1797
Epoch 2/20: - Loss = 3.1068
Epoch 3/20: - Loss = 2.3063
Epoch 4/20: - Loss = 1.6966
Epoch 5/20: - Loss = 1.2992
Epoch 6/20: - Loss = 1.0515
Epoch 7/20: - Loss = 0.8726
Epoch 8/20: - Loss = 0.7550
Epoch 9/20: - Loss = 0.6384
Epoch 10/20: - Loss = 0.5744
Epoch 11/20: - Loss = 0.5012
Epoch 12/20: - Loss = 0.4581
Epoch 13/20: - Loss = 0.4148
Epoch 14/20: - Loss = 0.3775
Epoch 15/20: - Loss = 0.3462
Epoch 16/20: - Loss = 0.3218
Epoch 17/20: - Loss = 0.3164
Epoch 18/20: - Loss = 0.2873
Epoch 19/20: - Loss = 0.2652
Epoch 20/20: - Loss = 0.2611
Epoch 1/20: - Loss = 3.1794
Epoch 2/20: - Loss = 3.1787
Epoch 3/20: - Loss = 3.1083
Epoch 4/20: - Loss = 2.4010
Epoch 5/20: - Loss = 1.8283
Epoch 6/20: - Loss = 1.4512
Epoch 7/20: - Loss = 1.1939
Epoch 8/20: - Loss = 1.0111
Epoch 9/20: - Loss = 0.8610
Epoch 10/20: - Loss = 0.7634
Epoch 11/20: - Loss = 0.6765
Epoch 12/20: - Loss = 0.5880
Epoch 13/20: - Loss = 0.5418
Epoch 14/20: - Loss = 0.4983
```